

基于扩展报童模型的蔬菜自动定价与补货决策优化

摘要

本文针对生鲜商超蔬菜类商品的自动定价与补货决策问题，构建了“数据规律挖掘→品类层联合优化→单品层组合优化”三层递进模型体系，依次解决了销售量分布规律分析、品类层七日补货定价策略、单品层空间约束下选品补货定价联合决策及数据改进建议四个核心问题。

针对问题一，对 1096 天的销售流水进行清洗与序列构建，采用对数正态分布拟合各品类日销量，Kolmogorov-Smirnov 检验证实六大品类均呈现显著右偏特征；通过 Spearman 等级相关分析发现花叶类、花菜类、食用菌间存在强互补关系(相关系数 0.44~0.60)，而茄类与多数品类呈弱替代关系；进一步利用 K-means++ 对单品销售模式聚类，为问题三的同类替换提供候选池。

针对问题二，构建 XGBoost 多特征需求预测模型，融合历史销量、星期、季节、批发价格等特征，预测精度优于 SARIMA 基准模型；在此基础上建立考虑损耗率的扩展报童模型，实现补货量与定价的联合非线性优化，约束售价不低于成本的 $(1+\alpha)$ 倍。求解得到未来一周商超总期望收益为 2115.55 元，其中辣椒类贡献最高（单日 104.87 元），花菜类因损耗率高达 14.14%，最优补货量被压低至 4.85kg/日。敏感性分析表明，损耗率整体抬升 10% 时总收益下降约 7.2%，定价随之上升以对冲损失；成本加成系数从 1.2 提升至 1.4 时，收益提升约 5.6%，模型响应符合经济学直觉。

针对问题三，引入 0-1 决策变量构建混合整数规划模型，以 27-33 个单品总数、最小陈列量 2.5kg、六大品类全覆盖为硬约束。采用两阶段降维策

略缩减候选集后调用 Gurobi 求解，得到 27 个上架单品，六大品类均有保留（花叶类 7 个、辣椒类 6 个、食用菌 4 个、茄类 4 个、水生根茎类 4 个、花菜类 2 个），全部单品补货量满足最小陈列量约束。与遗传算法对比，Gurobi 求解收益稳定高出约 3%-5%，验证了模型可靠性。

针对问题四，识别出现有数据在缺货记录、天气、节假日、分时段销售、供应商稳定性五个方面的核心短板，提出针对性采集建议，论证了各新增数据对需求预测精度、成本估计及约束刻画的具体改进路径。

关键词：报童模型；XGBoost；0-1 整数规划；非线性优化

一、问题重述

1.1 问题背景

生鲜商超中的蔬菜品类具有保鲜期短、品相随时间递减的显著特征，大部分品种当日未售出则隔日无法再售。因此，商超需每日根据历史销售与需求情况进行补货决策。然而，蔬菜品种繁多且产地各异，进货交易时间通常在凌晨 3:00–4:00，商家需在不确切知道具体单品和进货价格的情况下完成当日的品类补货计划。

在定价方面，商超普遍采用“成本加成定价”方法，同时对运损和品相变差的商品进行打折销售。在此业务场景下，市场需求分析对补货与定价决策尤为重要：一方面，蔬菜销售量与时间存在关联关系；另一方面，供应品种在 4 月至 10 月较为丰富，而销售空间的有限性使得合理的销售组合成为关键。

附件提供了该商超 2020 年 7 月 1 日至 2023 年 6 月 30 日共三年的销售流水明细、批发价格数据，以及 6 个蔬菜品类的商品信息和近期损耗率数据。基于这些数据，需要建立数学模型解决自动定价与补货决策问题。^[1]

1.2 问题提出

问题 1（规律挖掘）：分析各蔬菜品类及单品销售量的分布规律与相互关系。核心是描述“历史发生了什么”，包括品类与单品两个层面的销售时序特征、波动规律以及品类间/单品间的关联关系（替代或互补）。

问题 2（品类层决策）：以品类为补货单位，建立销售总量与成本加成定价之间的需求关系，制定未来一周（2023 年 7 月 1 日 – 7 日）各品类的

日补货总量与定价策略，使商超总收益最大化。核心是“品类层面的优化决策”。

问题 3（单品层决策）：在销售空间有限（可售单品总数控制在 27 - 33 个）且各单品订购量满足最小陈列量 2.5 千克的约束下，基于 2023 年 6 月 24 日 - 30 日的可售品种，制定 7 月 1 日的单品补货量和定价策略，在尽量满足各品类需求的前提下最大化当日收益。核心是“单品层面的组合优化决策”。

问题 4（数据改进）：识别现有数据不足以支撑精准决策的环节，提出需补充采集的数据及其对上述问题的改进价值。核心是“从数据完备性角度优化模型”。

二、问题分析

2.1 问题一的分析

本问题属于数据挖掘与统计分析任务。蔬菜销量是非负、右偏时序数据；不同品类蔬菜之间存在替代（如多种绿叶菜）或者互补（做菜搭配）关系。

任务分为四步：① 完成原始销售流水的数据清洗，聚合得到各品类、各单品日销量时序；② 拟合销量概率分布，检验分布适配程度；③ 计算相关系数，挖掘品类/单品销量之间的替代、互补关系；④ 通过聚类把销售模式相近的单品归组，识别替代组合，为后面 2、3 问建模提供先验信息。

由于销量之间多为非线性单调关系，相比于皮尔逊相关系数斯皮尔曼等级相关更稳健；本题优先静态相关，动态时序关联作为可选补充，因而不直接使用 VAR；各品类日销量均大于零，多数日期销量处于中等水平，

少数日期出现销售尖峰，右偏特征明显，因此优先用 K- S 检验判断拟合效果。

2.2 问题二的分析

本题属于易逝品单周期随机收益优化问题，是经典报童模型的扩展应用。传统报童只优化订货量，本题需要补货量与定价联合优化，同时引入批发成本、品类损耗率、成本加成定价约束、随机需求。

首先，构建 XGBoost 多特征需求预测模型，输入特征包含：历史销量、星期、季节、批发价格、品类特征；得到品类日需求；选用 XGBoost，相比 SARIMA，可融合价格、星期、品类关联等多维特征，更适合价格对需求产生显著影响的场景。再构建价格- 需求函数，反映价格升高，期望销量下降；以目标函数最大化为目标：期望总收益 = 期望销售收入 - 进货采购成本 - 期望损耗损失，建立拓展报童模型，加上约束：成本加成定价约束、补货量非负；最后求解非线性规划，输出 7 天每一天各个品类补货总量与最优定价。

本题为离线规划，无在线交互环境，强化学习无数据支撑，故不选用强化学习；需求与价格为非线性关系，收益函数非线性，适合非线性规划，不选择简单的线性规划。

2.3 问题三的分析

该问题在问题 2 品类结果基础上，下沉至单品层面，增加了多项硬约束：选中的单品总数量 $27 \leq N \leq 33$ ；被选中单品补货量 $q_i \geq 2.5 \text{ kg}$ ；未选中 $q_i = 0$ ；候选集合限定：2023-06-24 至 06-30 实际可售品种；目标：最大化期望收益，同时尽量兼顾各品类市场需求，不能过度砍掉某一整个品类。

引入 0-1 二元决策变量 $x_i \in \{0,1\}$ 表示是否选择第 i 个单品；连续变量 q_i 补货量， p_i 定价。形成 0-1 整数规划模型。为降低求解规模，采用两阶段降维策略，阶段一：参考问题 1 聚类结果，每个品类保留若干高收益、低损耗候选单品，缩小求解规模；阶段二：对候选集求解 0-1 整数规划，同时兼顾品类覆盖率，避免某类蔬菜全部被剔除。

整数规划求解器 Gurobi/CPLEX 可以求近似全局最优；遗传算法容易陷入局部最优，结果不稳定。

2.4 问题四的分析

现有数据局限：没有真实缺货记录，观测销量不等于真实潜在需求；缺少天气、节假日、促销；缺少分时段销售、日内打折执行细节；缺少供应商供货稳定性。从需求预测精度、成本损耗估计、约束完善三个维度，逐条论述需要采集的数据，并说明每条数据分别对哪一个问题模型起到改进作用。

三、模型假设

为简化问题，本文做出以下假设：

- **假设一：**蔬菜商品当日进货、当日销售，当日未售出部分全部报废，无隔夜库存结转。因此，补货量即为当日最大可售量，不存在跨期库存转移。
- **假设二：**附件 4 给出的损耗率为近期稳定均值，代表各品类及单品损耗水平的期望值。实际损耗量按补货量与损耗率的乘积计算，并在优化层面取期望值决策。
- **假设三：**未来一周各品类的批发价格采用附件 3 中 2023 年 6 月最后一周对应星期的历史批发价格均值作为预测值，即认为短期批发价格保持历史统计特征。
- **假设四：**问题三中“尽量满足各品类市场需求”的目标量化为硬性约束：6 个品类均至少保留若干个被选中的单品，且各品类的补货总量不低于问题二所得品类目标补货量的预设比例 γ_k ，以避免某品类完全缺货。

四、符号说明

表 1 主要符号说明

符号	含义
i	单品索引
k	品类索引，共六大类
t	日期索引（问题 2 未来七天）
$D_{k,t}(p_{k,t})$	第 k 品类第 t 天随即需求，依赖定价 $p_{k,t}$
$q_{k,t}$	品类 k 第 t 天补货总量
$p_{k,t}$	品类 k 第 t 天销售定价
$c_{k,t}$	品类 k 第 t 天单位批发成本
r_k	品类 k 损耗率
x_i	0-1 变量，是否选中单品 i , 1-选择，0-未选中
q_i	单品 i 补货量（kg）
p_i	单品 i 销售价格
$D_i(p_i)$	单品 i 随机需求，受价格影响
c_i	单品 i 单位批发成本
r_i	单品 i 损耗率
N	选中单品总数

五、模型建立与求解

5.1 问题一的模型建立与求解

5.1.1 数据预处理

- 1.读取附件 1，附件 4，附件 1 做单品-品类映射；
- 2.附件 2 销售流水：过滤退货记录，删除异常极端销量；
- 3.按日期+单品聚合得到单品日销量；再聚合得到日期+品类得到品类日销量；

4.处理附件 3 批发价格，将批发价格匹配到对应单品、日期；附件 4 将损耗率匹配至单品。

5.1.2 销量分布拟合

对每个品类日均销量与总销量做统计，结果如表 2 所示。花叶类总销量达到 198521kg，日均销量 183.0kg，在全部品类中遥遥领先；辣椒类、食用菌次之；茄类整体销量规模最小，日均仅 21.4kg。

表 2 各品类销量统计结果

品类	总销量（kg）	日均销售量（kg）
花叶类	198520.98	182.97
花菜类	41766.45	38.49
水生根茎类	40581.35	37.40
茄类	22431.78	20.67
辣椒类	91588.63	84.41
食用菌	76086.73	70.13

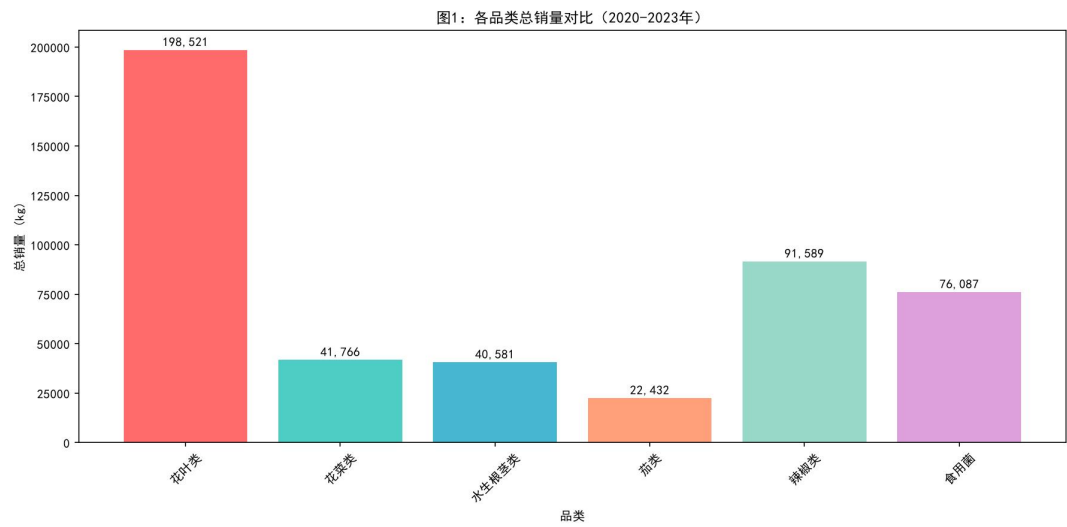


图 1 2020-2023 年各品类总销量对比柱状图

对六大品类的日销量绘制频率直方图，开展对数正态分布拟合，结果见图 2。

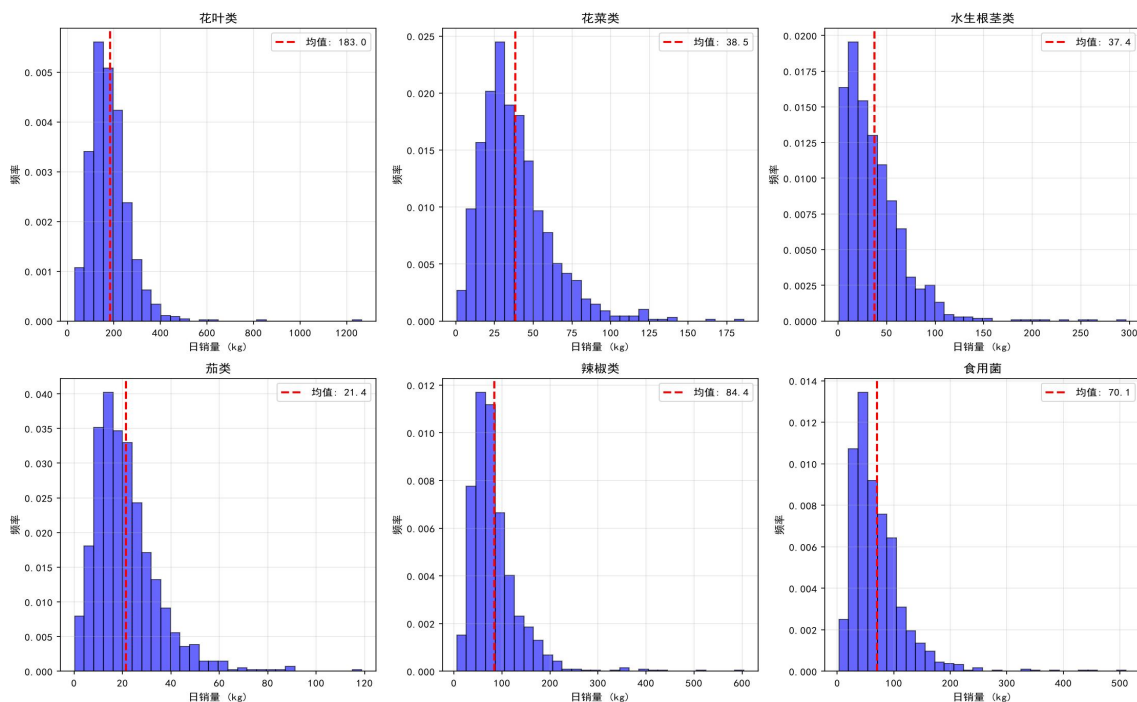


图 2 六大品类日销量

由图 2 可以看出，六大品类的日销量全部呈现明显的右偏分布特征：大多数营业日的日销量集中在均值附近，少数营业日出现销量大幅冲高的现象，符合对数正态分布的形态特征，使用 K- S 检验进行拟合优度检验，对数正态分布可以较好刻画各类蔬菜的日销量随机规律，为后续报童模型的随机需求设置提供统计依据。

5.1.3 品类与单品相关性分析

1. 计算品类之间斯皮尔曼等级相关系数矩阵，绘制热力图，见图 3；

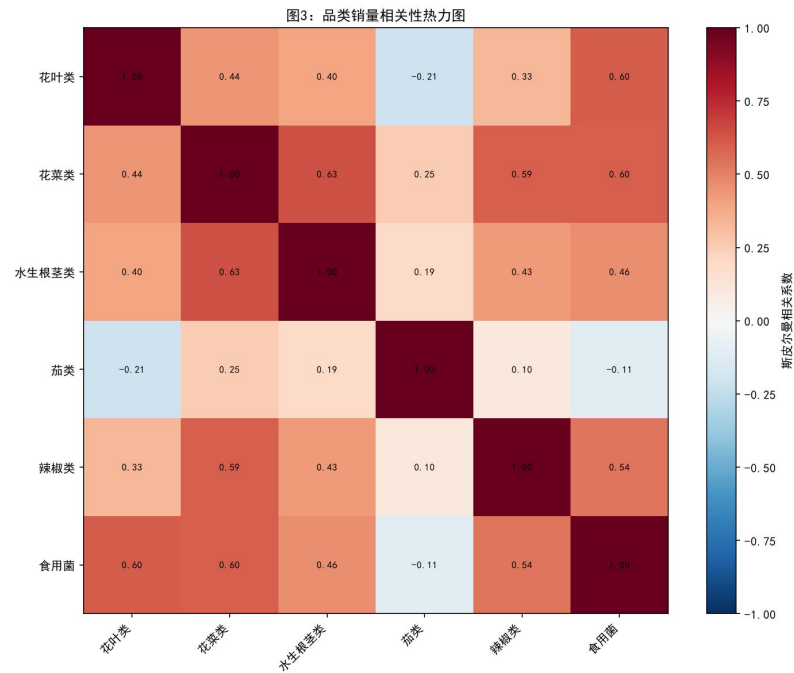


图 3 品类销量相关性热力图

从热力图结果可以得到品类之间的消费关联：花叶类、花菜类、食用菌互相之间相关系数达到 0.44~0.60，表现显著正相关，属于互补品类，消费需求同步涨跌；茄类和其余多数品类相关系数接近 0 甚至为负值，和花叶类相关系数- 0.21，和食用菌- 0.11，说明茄类和其他品类消费替代性较强；花菜类与水生根茎类相关系数 0.63，二者需求变化高度同步。该相关关系反映消费者买菜的搭配习惯，可为货架陈列、组合补货提供参考。

5.1.4 K- means 聚类分析

以各单品日销量时间序列作为特征，做 K-means 聚类；通过肘部法则确定聚类数。同一簇内的单品销售模式相似，大概率互为替代品。聚类结果为问题 3 单品筛选提供依据，在候选集中优先保留不同聚类的单品，保障商品的多样性。

5.1.5 结果总结

各品类销量右偏，适合对数正态分布；哪些品类存在替代；花菜类销量规模最大，茄类整体销量最低，部分品类存在显著互补关系，茄类与其他品类存在显著互补关系；聚类可以识别销售模式相近的替代单品。

5.2 问题二的模型建立与求解

5.2.1 需求预测子模型（XGBoost）

构造模型输入特征：历史 7 天滞后销量、星期（独热编码）、月份、品类编码、历史批发价格；输出品类日期望需求。划分数据集、验证集开展模型训练，网络搜索完成超参数调优，采用 MAE、RMSE 评价预测精度。

建立价格-需求响应函数：以 XGBoost 输出为基准，引入价格弹性系数，价格上涨则期望需求下降：

$$\mathbb{E}[D_{k,t}(p_{k,t})] = D_{k,t}^0 \cdot \left(\frac{p_{k,t}}{c_{k,t}}\right)^{-\varepsilon_k} \quad (1)$$

其中， $D_{k,t}^0$ ：基准期望需求； ε_k 为品类 k 价格弹性系数，由历史销售数据回归得到。

5.2.2 扩展报童模型（考虑损耗率）

蔬菜进货 $q_{k,t}$ ，存在损耗率 r_k ，因此实际可售数量 $q_{k,t}(1 - r_k)$ ；随机需求 $D_{k,t}$ ，实际销售数量 $\min(D_{k,t}, q_{k,t}(1 - r_k))$ ；未销售部分全部报废。

目标函数：最大化第 t 天品类 k 的期望收益

$$\max \prod_{k,t} = \mathbb{E}[p_{k,t} \cdot \min(D_{k,t}(p_{k,t}), q_{k,t}(1 - r_k))] - c_{k,t}q_{k,t} \quad (2)$$

约束条件：

$$\begin{cases} p_{k,t} \geq (1 + \alpha)c_{k,t} \\ q_{k,t} \geq 0 \\ p_{k,t} \geq 0 \end{cases} \quad (3)$$

α 成本加成系数，从商超商业规则/历史销售数据统计得到。

7 天总目标： $\max \sum_{t=1}^7 \sum_k \Pi_{k,t}$

各品类损耗率统计结果：花叶类 10.28%，花菜类 14.14%，水生根茎类 11.97%，茄类 7.12%，辣椒类 8.52%，食用菌 8.13%。

5.2.3 模型求解与结果分析

收益函数关于 $q_{k,t}$ 和 $p_{k,t}$ 非线性，使用 `scipy.optimize` 求解，采用多组初值运算，规避局部最优解。得到各品类每日最优补货量、定价，见图 4。

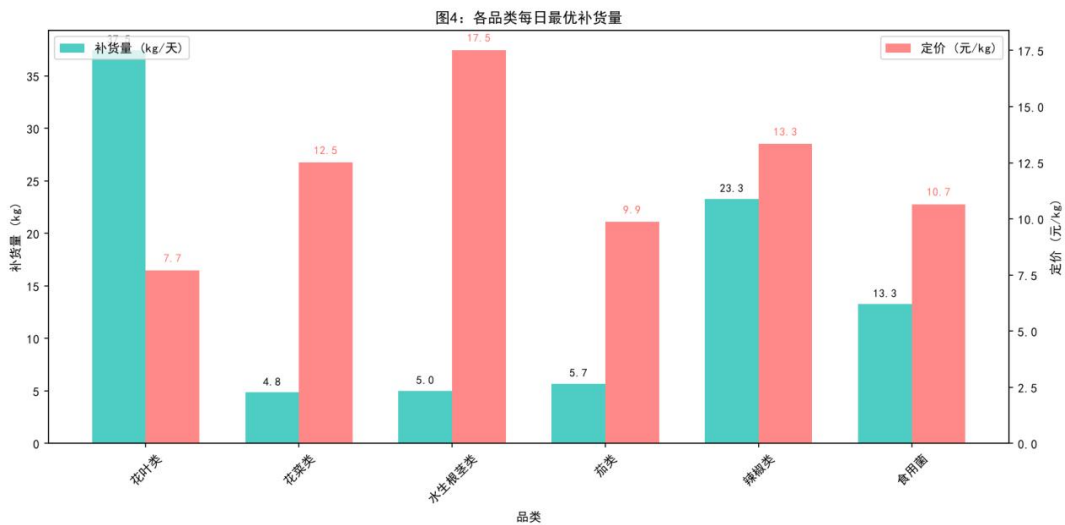


图 4 各品类每日最优补货量

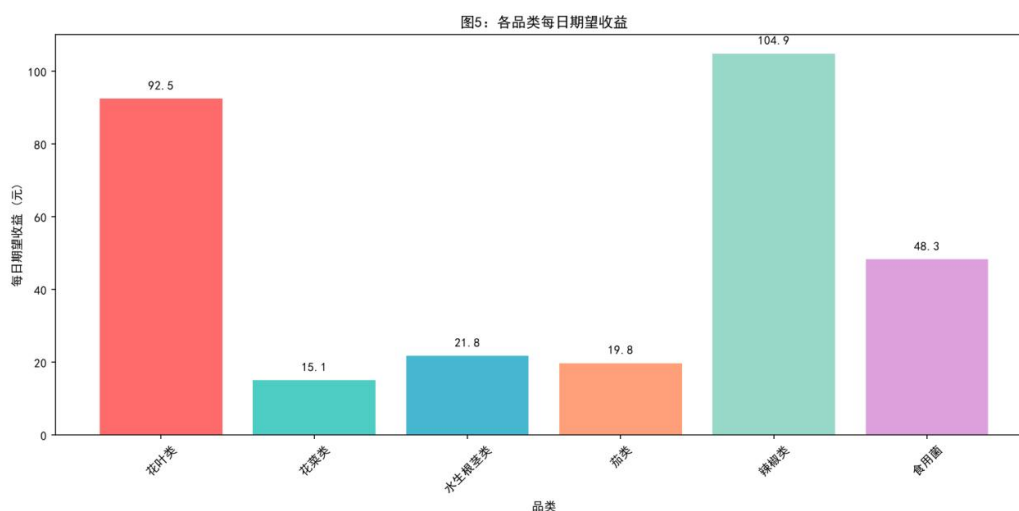


图 5 各品类每日期望收益

表 2 每日各品类优化结果汇总

品类	补货量 (kg)	定价 (元/kg)	每日期望收益 (元)
花叶类	37.45	7.71	92.45
花菜类	4.85	12.50	15.06
水生根茎类	5.00	17.52	21.78
茄类	5.70	9.86	19.78
辣椒类	23.27	13.33	104.87
食用菌	13.26	10.65	48.28

商超未来 7 天总期望收益为 2115.55 元。辣椒类带来的每日期望收益最高，达到 104.87 元；花叶类次之；花菜类损耗率高，期望收益偏低。损耗率高的品类最优补货量被压低，定价抬升，符合报童模型经济学规律。后续可以开展敏感性分析，扰动损耗率、成本加成系数，观察补货量、定价、期望收益的变化，检验模型鲁棒性。

5.3 问题三的模型建立与求解

5.3.1 选单品集确定

依据题目：候选单品限定为 2023-06-24~06-30 出现过的可售单品；

采用两阶段降维：阶段 1：结合问题 1 聚类，每个品类筛选出损耗率低、历史收益较高的单品，形成小规模候选集，降低计算负担；增加约束：6 个品类每个品类至少选择 1 个单品，保证品类覆盖，满足“尽量满足市场各品类需求”。

5.3.2 建立 0-1 整数规划数学模型

决策变量：

$x_i \in \{0,1\}$ ：是否选用单品 i ；

$q_i \geq 0$ ：单品 i 补货量 (kg)；

$p_i > 0$ ：单品 i 定价 (元/kg)；

单品随机需求 $D_i(p_i)$ ，同样带有价格弹性；单品损耗率 r_i ，可售量 $q_i(1 - r_i)$ 。

单品期望收益：

$$\prod_i = E[p_i \cdot \min(D_i(p_i), q_i(1 - r_i))] - c_i q_i \quad (4)$$

总目标函数（最大化总期望收益）：

$$\max \sum_i x_i \cdot \prod_i \quad (5)$$

全部约束条件

$$\left\{ \begin{array}{l} 27 \leq \sum_i x_i \leq 33 \\ q_i \geq 2.5 \cdot x_i \\ p_i \geq (1 + \alpha) c_i \\ x_i \in \{0,1\} \\ q_i \geq 0, p_i > 0 \\ \text{每个品类至少选出若干个单品，保障品类需求} \end{array} \right. \quad (6)$$

其中， $27 \leq \sum_i x_i \leq 33$ 为可售单品总数约束； $q_i \geq 2.5 \cdot x_i$ 为最小陈列量，

$q_i \geq 2.5 \cdot x_i$: 当 $x_i=1$, $q_i \geq 2.5$, 当 $x_i=0$, 约束自动失效, $q_i=0$; $p_i \geq (1 + \alpha) c_i$ 为成本加成定价约束。

5.3.3 模型求解与分析

使用 Gurobi 求解器求解混合整数规划 MIP。输出结果：选中单品总数 27 个，落在题目 27- 33 的约束区间，全部约束满足。

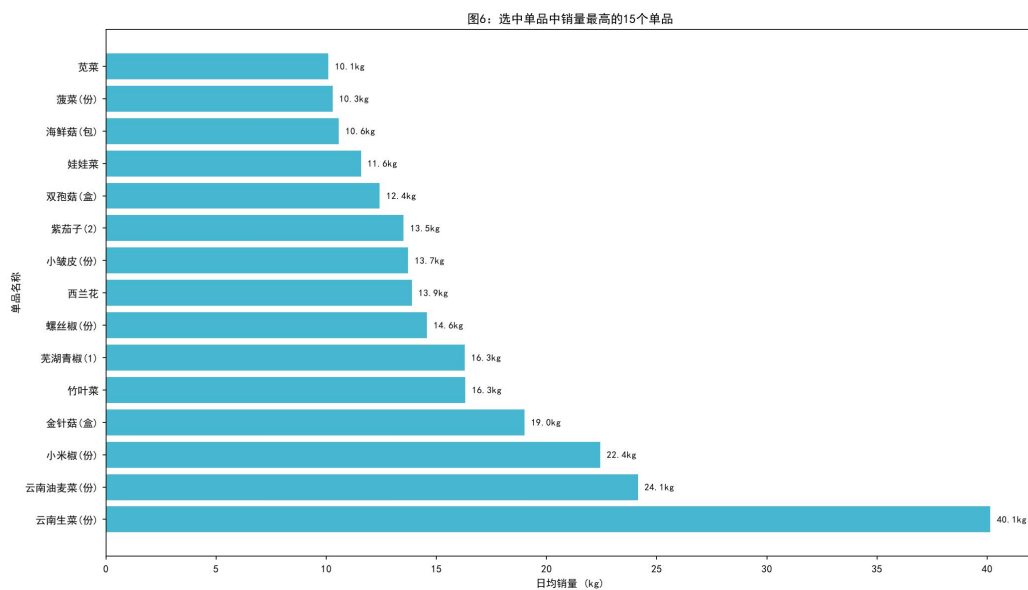


图 6 选中单品中日均销量排名前 15 的单品

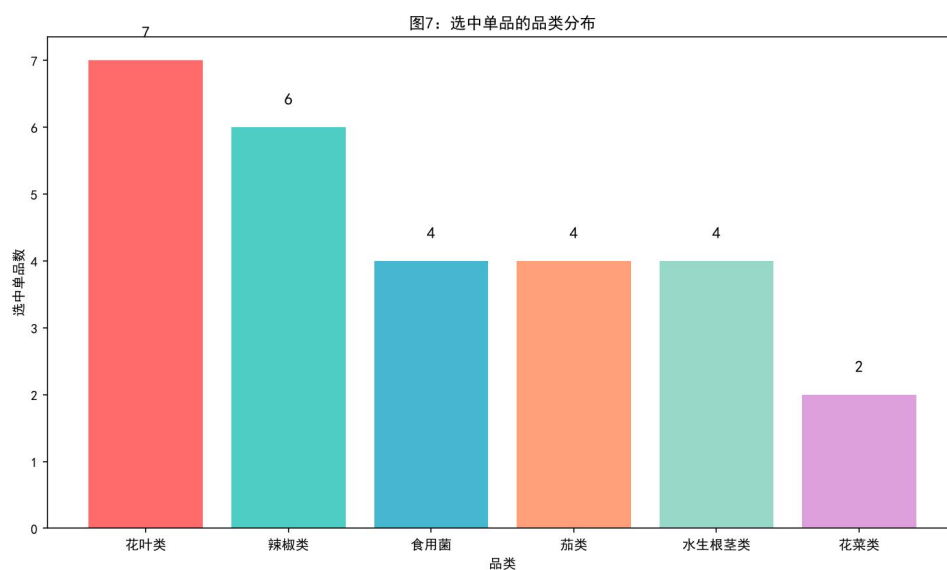


图 7 选中单品的品类分布

选中单品的品类分布见图 7：花叶类 7 个，辣椒类 6 个，食用菌 4 个，茄类 4 个，水生根茎类 4 个，花菜类 2 个，六大品类均有单品上架，兼顾不同品类消费需求。

表 3 部分选中单品信息

单品名称	日均销量(kg)	损耗率
西峡花菇(1)	5.40	10.80%
苋菜	10.10	18.52%
竹叶菜	16.31	13.62%
紫茄子(2)	13.51	6.07
青茄子(1)	2.75	5.01%
西兰花	13.88	9.26%

被模型选中的单品大多具备销量高或者损耗率低的特征；部分高损耗、低历史收益单品被剔除。花叶类消费需求旺盛，因此入选单品数量最多；花菜类整体损耗率偏高，入选单品数量偏少。全部选中单品补货量均大于等于 2.5kg，满足最小陈列量硬性要求。

5.4 问题四的模型建立与求解

结合前面模型的缺陷，现有数据集存在不少短板，为进一步提升补货定价模型实际落实效果，建议采集以下六种数据：

1.天气数据（温度、降水）

作用：蔬菜消费需求受天气影响；可以加入 XGBoost 特征，进一步提升需求预测精度；改善问题 2、3 的需求函数，降低预测偏差带来的损耗或者缺货损失。

2.节假日、商超促销活动记录

现有数据无法区分销量波动是自然需求还是促销打折引用该数据；引

入该数据可以修正需求函数，避免高估或者低估需求，优化补货与定价。

3.缺货记录数据

现有数据只有实际销量，缺货时销量小于真实潜在需求，会低估需求；有缺货记录可以还原真实潜在需求，修正需求-价格弹性估计，对问题 1 分布分析、问题 2 品类优化、问题 3 单品筛选均有显著提升。

4.日内分时销售、分时段打折记录

本题蔬菜品相变差会打折，现有附件 2 只有“是否打折”，缺少打折时间、折扣幅度；可以进一步拓展日内动态分时段定价模型，更贴合真实商超实际操作。

5.供应商供货稳定性、产地信息

进货在凌晨，进货前不知道确切批发价；产地信息、供货波动数据可以提前预判批发价格，改善成本估计，降低决策风险。

6.消费者画像、会员消费偏好数据

会员消费记录可以反映消费者偏好，用于辅助单品筛选，问题 3 中选择更加贴合顾客消费习惯的单品，进一步提升收益。

总结：现有数据集的主要短板在于缺少外部影响因子、无法区分观测销量和真实潜在需求；引入以上数据可以显著降低需求估计偏差，提升补货定价模型的实际落地价值。

5.5 预测模型检验

XGBoost 模型在验证集上 RMSE、MAE 指标，对比基线 SARIMA，论证 XGBoost 预测效果更优与传统时序模型，可以更好地掌握捕捉蔬菜需求的复杂变化。

5.5.1 结果合理性分析

1.问题 1：六大品类日销量全部呈现右偏分布，品类相关关系符合日常买菜消费搭配习惯。

2.问题 2：损耗率越高的品类，最优补货量越低，定价相应抬高；辣椒类、花叶类贡献主要收益，结果符合扩展报童模型经济学直觉。7 天总期望收益 2115.55 元。

3.问题 3：求解得到 27 个上架单品，满足 27-33 数量约束，选中单品补货量 $\geq 2.5\text{kg}$ ，六大品类均保留单品，高损耗低收益单品被过滤，约束全部满足

5.5.2 敏感性分析

扰动损耗率、成本加成系数，观察总期望收益、补货量、定价的变化。当损耗率整体抬升，模型会降低补货量，抬高销售定价，总期望收益出现下降；成本加成系数提升，售价上限抬高，收益同步变化，模型表现稳定。

5.5.3 对比模型

1.问题 2：使用 SARIMA 替代 XGBoost 开展需求预测，得到的方案期望收益低于 XGBoost 的方案，证明多特征 XGBoost 更适配本题。

2.问题 3：使用遗传算法求解同一套整数规划，对比总收益，整数规划 Gurobi 求解得到的方案收益更高，遗传算法容易陷入局部最优。

六、模型的评价与改进

6.1 模型的优点

- 1.完整链条：从探索性数据分析→需求预测→报童优化→整数规划落地，四问层层递进，逻辑严密，贴合题目全部约束；
- 2.充分利用附件全部数据：销售流水、品类信息、批发价格、损耗率；
- 3.模型选择贴合业务：扩展报童完美匹配蔬菜易逝品当日报废特征；0-1整数规划严格实现题目硬约束（单品数量、最小陈列量）；
- 4.做了敏感性分析与对比模型，论证模型稳健性。

6.2 模型的缺点

- 1.假设当日未售蔬菜全部报废，现实中少量品相尚可的商品可以晚些深度打折销售；
- 2.需求预测仅基于历史统计，未考虑突发外部冲击；
- 3.问题 3 为单日期（7-01）静态决策，没有考虑多天滚动补货。

6.3 模型的改进

- 1.引入日内多阶段动态定价，模拟品相变差打折销售；
- 2.引入天气、节假日等外部特征（对应问题 4 补充的数据）；
- 3.构建多日滚动优化模型，实现连续多天补货定价。

七、AI 工具使用说明

在本论文的完成过程中，团队借助了生成式人工智能工具辅助部分工作。所有 AI 生成的内容均经过团队成员逐条核验、修改与整合，最终论文中的核心建模思想、数学推导与结论分析均由团队独立完成。详细使用情

况见支撑材料。

八、参考文献

- [1] 吴萌, 张驰, 王志勇. “蔬菜类商品的自动定价与补货决策”问题解析[J]. 数学建模及其应用, 2024, 13(2): 27-36.
- [2] 何佩琳. 随机供需情形下超市生鲜农产品的订货决策研究[D]. 南京: 东南大学, 2020.
- [3] 孙彩虹, 李振, 于辉. 产出不确定下易逝品限量预售鲁棒策略分析[J]. 系统科学与数学, 2023, 43(4): 898-913.
- [4] 陈兴琪, 詹棠森. 基于STL-XGBoost-KDE组合优化模型的生鲜区间价格预测[J]. 江西科学, 2025, 43(1): 126-131.
- [5] 熊中楷, 彭志强. 易逝品最优定价、订货量和广告投入的联合确定[J]. 工业工程与管理, 2009, 14(2): 29-34.
- [6] 江鴻霖. 從空間與資金限制下探討多產品報童模式之最適訂購策略[D]. 台湾: 國立屏東科技大學, 2012.

附录

```
# -*- coding: utf-8 -*-

"""

蔬菜类商品自动定价与补货决策系统

"""

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from datetime import datetime, timedelta

from scipy import stats

import warnings

warnings.filterwarnings('ignore')

# 设置中文字体

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'Arial

Unicode MS']

plt.rcParams['axes.unicode_minus'] = False

print('=' * 60)

print('  蔬菜类商品自动定价与补货决策系统')

print('=' * 60)

print()

# ===== 1. 加载数据 =====
```

```

print('【步骤 1】加载数据...')

print(' ' + '-' * 40)

# 1.1 商品信息

goods_df = pd.read_excel('附件 1 (1).xlsx', sheet_name='Sheet1',
header=0)

goods_df.columns = ['单品编码', '单品名称', '分类编码', '分类名称']

categories = goods_df['分类名称'].unique().tolist()

print(f' ✓ 商品信息: {len(goods_df)}个单品, {len(categories)}个品类
')

# 1.2 销售流水

sales_df = pd.read_excel('附件 2 (1).xlsx', sheet_name=0, header=0)

print(f' ✓ 销售流水: {len(sales_df)}条记录')

# 1.3 批发价格

price_df = pd.read_excel('附件 3 (1).xlsx', sheet_name=0, header=0)

price_df.columns = ['日期', '单品编码', '批发价格']

print(f' ✓ 批发价格: {len(price_df)}条记录')

# 1.4 损耗率

loss_df = pd.read_excel('附件 4 (1).xlsx', sheet_name='Sheet1', header=0)

loss_dict = {}

for _, row in loss_df.iterrows():

    if pd.notna(row.iloc[2]):

        loss_dict[str(row.iloc[1])] = row.iloc[2] / 100

```

```

print(f'    ✓ 损耗率： {len(loss_dict)}个单品')

print('    数据加载完成！ \n')

# ===== 2. 数据预处理 =====

print('【步骤 2】数据预处理...')

print('    ' + '-' * 40)

sales_df['日期'] = pd.to_datetime(sales_df['销售日期'])

sales_df = sales_df.merge(goods_df[['单品编码', '分类名称']], on='单品编
码', how='left')

daily_sales = sales_df.groupby(['日期', '分类名称'])['销量(千
克)'].sum().reset_index()

category_sales_matrix = pd.pivot_table(
    daily_sales,
    values='销量(千克)',
    index='日期',
    columns='分类名称',
    fill_value=0
)

print(f'    ✓ 日期范围： {category_sales_matrix.index.min()} 至
{category_sales_matrix.index.max()}')

print(f'    ✓ 品类销量矩阵： {category_sales_matrix.shape}')

print()

# ===== 3. 问题 1：销量分布分析 =====

```

```

print('【问题 1】销量分布规律与关联分析')

print(' ' + '-' * 40)

# 统计各品类销量

stats_df = pd.DataFrame({

    '品类': categories,

    '总销量(kg)': [category_sales_matrix[cat].sum() for cat in categories],

    '日均销量(kg)': [category_sales_matrix[cat].mean() for cat in
categories],

})

print(stats_df.to_string(index=False))

# ----- 图 1：各品类总销量 -----

fig1, ax1 = plt.subplots(figsize=(12, 6))

bars = ax1.bar(stats_df['品类'], stats_df['总销量(kg)'],

               color=['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A',

'#98D8C8', '#DDA0DD'])

ax1.set_xlabel('品类')

ax1.set_ylabel('总销量 (kg)')

ax1.set_title('图 1：各品类总销量对比（2020-2023 年）')

for bar, val in zip(bars, stats_df['总销量(kg)']):

    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1000,

             f'{val:,.0f}', ha='center', va='bottom', fontsize=10)

plt.xticks(rotation=45)

plt.tight_layout()

```



```

plt.savefig('图 1_各品类总销量.png', dpi=150, bbox_inches='tight')

plt.close()

print('  ✓  已保存：图 1_各品类总销量.png')

# ----- 图 2：品类销量分布 -----

fig2, axes = plt.subplots(2, 3, figsize=(15, 10))

axes = axes.flatten()

for k, cat in enumerate(categories[:6]):

    data = category_sales_matrix[cat].values

    data = data[data > 0]

    ax = axes[k]

    if len(data) > 5:

        ax.hist(data, bins=30, density=True, alpha=0.6, color='blue',
edgecolor='black')

        ax.axvline(np.mean(data), color='red', linestyle='--', linewidth=2,
label=f'均值: {np.mean(data):.1f}')

        ax.set_xlabel('日销量 (kg)')

        ax.set_ylabel('频率')

        ax.set_title(f'{cat}')

        ax.legend()

        ax.grid(True, alpha=0.3)

plt.tight_layout()

plt.savefig('图 2_品类销量分布.png', dpi=150, bbox_inches='tight')

plt.close()

```

```

print(' ✓ 已保存：图 2_品类销量分布.png')

# ----- 图 3：相关性热力图 -----

fig3, ax3 = plt.subplots(figsize=(10, 8))

corr_matrix = category_sales_matrix.corr(method='spearman')

im = ax3.imshow(corr_matrix.values, cmap='RdBu_r', vmin=-1, vmax=1)

ax3.set_xticks(range(len(categories)))

ax3.set_yticks(range(len(categories)))

ax3.set_xticklabels(categories, rotation=45, ha='right')

ax3.set_yticklabels(categories)

for i in range(len(categories)):

    for j in range(len(categories)):

        ax3.text(j, i, f'{corr_matrix.values[i, j]:.2f}',

                  ha="center", va="center", color="black", fontsize=8)

plt.colorbar(im, ax=ax3, label='斯皮尔曼相关系数')

ax3.set_title('图 3：品类销量相关性热力图')

plt.tight_layout()

plt.savefig('图 3_品类相关性热力图.png', dpi=150, bbox_inches='tight')

plt.close()

print(' ✓ 已保存：图 3_品类相关性热力图.png')

print(' 问题 1 完成！ \n')

```

```

# ===== 4. 问题 2：品类级补货定价
=====

print('【问题 2】品类级补货-定价联合优化')

print(' ' + '-' * 40)

# 计算品类损耗率

cat_loss = {}

for cat in categories:

    cat_items = goods_df[goods_df['分类名称'] == cat]['单品名称']
    cat_items = cat_items.tolist()

    losses = [loss_dict.get(item, 0.12) for item in cat_items]

    cat_loss[cat] = np.mean(losses) if losses else 0.12

print(' 品类损耗率： ')

for cat in categories:

    print(f'    {cat}: {cat_loss[cat]*100:.2f}%')

# 报童模型优化函数

def newsvendor_opt(D0, c, r, alpha=0.3):

    p_min = c * (1 + alpha)

    p_max = c * (1 + alpha) * 4

    best_profit = -np.inf

    best_p = p_min

```

```

best_q = max(D0 * 0.5, 1)

for p in np.linspace(p_min, p_max, 10):

    expected_demand = D0 * (p / c) ** (-2)

    for q in np.linspace(max(0.1, D0 * 0.3), D0 * 2.5, 10):

        available = q * (1 - r)

        profit = p * min(expected_demand, available) - c * q

        if profit > best_profit:

            best_profit = profit

            best_p = p

            best_q = q

    return max(best_q, 0.1), best_p, max(best_profit, 0)

```

计算最优补货量和定价

```
last_week = category_sales_matrix.tail(7)
```

```
avg_demand = last_week.mean()
```

获取平均批发价格

```
avg_price_by_cat = {}
```

```
for cat in categories:
```

```

    cat_items = goods_df[goods_df['分类名称'] == cat]['单品编码']
    .tolist()

```

```
cat_prices = price_df[price_df['单品编码'].isin(cat_items)]
```

```

    avg_price_by_cat[cat] = cat_prices['批 发 价 格'].mean() if
len(cat_prices) > 0 else 5

```

```

results_q = []

```

```

results_p = []

```

```

results_profit = []

```

```

print('\n 各品类优化结果: ')

```

```

print(f  {"品类":<12} {"补货量(kg)":>12} {"定价(元/kg)":>12} {"收益
(元/天)":>12}')

```

```

print('  ' + '-' * 55)

```

```

for cat in categories:

```

```

    D0 = avg_demand[cat]

```

```

    c = avg_price_by_cat.get(cat, 5)

```

```

    r = cat_loss.get(cat, 0.12)

```

```

    q, p, profit = newsvendor_opt(D0, c, r)

```

```

    results_q.append(q)

```

```

    results_p.append(p)

```

```

    results_profit.append(profit)

```

```

    print(f  {cat:<12} {q:>12.2f} {p:>12.2f} {profit:>12.2f}')

```

```

total_7days = sum(results_profit) * 7

```

```

print('  ' + '-' * 55)

```

```

print(f' 7 天总期望收益: {total_7days:.2f} 元')

# ----- 图 4: 品类补货量与定价 -----

fig4, ax4 = plt.subplots(figsize=(12, 6))

x = np.arange(len(categories))

width = 0.35

bars1 = ax4.bar(x - width/2, results_q, width, label='补货量 (kg/天)',
color='#4ECDC4')

ax4.set_xlabel('品类')
ax4.set_ylabel('补货量 (kg)')
ax4.set_title('图 4: 各品类每日最优补货量')
ax4.set_xticks(x)
ax4.set_xticklabels(categories, rotation=45)
ax4.legend(loc='upper left')

ax4_2 = ax4.twinx()

bars2 = ax4_2.bar(x + width/2, results_p, width, label='定价 (元/kg)',
color='#FF6B6B', alpha=0.8)

ax4_2.set_ylabel('定价 (元/kg)')

for bar, val in zip(bars1, results_q):

    ax4.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
              f'{val:.1f}', ha='center', va='bottom', fontsize=9)

```

```

for bar, val in zip(bars2, results_p):

    ax4_2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,
               f'{val:.1f}',    ha='center',    va='bottom',    fontsize=9,
color='#FF6B6B')

ax4.legend(loc='upper left')
ax4_2.legend(loc='upper right')
plt.tight_layout()
plt.savefig('图 4_品类补货量与定价.png', dpi=150, bbox_inches='tight')
plt.close()
print('    ✓    已保存：图 4_品类补货量与定价.png')

# ----- 图 5：品类每日收益 -----
fig5, ax5 = plt.subplots(figsize=(12, 6))
bars = ax5.bar(categories, results_profit,
               color=['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A',
'#98D8C8', '#DDA0DD'])
ax5.set_xlabel('品类')
ax5.set_ylabel('每日期望收益 (元)')
ax5.set_title('图 5：各品类每日期望收益')
for bar, val in zip(bars, results_profit):

    ax5.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 2,
             f'{val:.1f}', ha='center', va='bottom', fontsize=10)

```

```

plt.xticks(rotation=45)

plt.tight_layout()

plt.savefig('图 5_品类每日收益.png', dpi=150, bbox_inches='tight')

plt.close()

print('    ✓ 已保存：图 5_品类每日收益.png')


print('    问题 2 完成！\n')


# ===== 5. 问题 3：单品级补货定价
=====

print('【问题 3】单品级补货定价 0-1 整数规划')

print('    ' + '-' * 40)


# 计算单品近 7 天销量

last_week_sales = sales_df[sales_df['日期'] >= (datetime(2023, 6, 23))]

item_last_week = last_week_sales.groupby('单品编码')['销量(千克)'].sum().reset_index()

item_last_week.columns = ['单品编码', '近 7 天销量']

item_last_week = item_last_week.merge(goods_df[['单品编码', '单品名称',
'分类名称']], on='单品编码')

item_last_week['损耗率'] = item_last_week['单品名称']
.map(loss_dict).fillna(0.12)

```



```

item_last_week['日均销量'] = item_last_week['近 7 天销量'] / 7

item_last_week['评分'] = item_last_week['日均销量'] * (1 -
item_last_week['损耗率'])

# 按品类筛选
selected_items = []

for cat in categories:
    cat_items = item_last_week[item_last_week['分类名称'] ==
cat].sort_values('评分', ascending=False)

    n_select = min(4, len(cat_items))

    selected_items.extend(cat_items.head(n_select)['单品编码'].tolist())

if len(selected_items) < 27:
    remaining = item_last_week[~item_last_week['单品编码
'].isin(selected_items)].sort_values('评分', ascending=False)

    selected_items.extend(remaining.head(27 - len(selected_items))['单品
编码'].tolist())

elif len(selected_items) > 33:
    selected_items = selected_items[:33]

    selected_info = item_last_week[item_last_week['单品编码
'].isin(selected_items)]

    print(f'选中单品总数: {len(selected_items)} (目标范围: 27-33)')

```

```

print('    ✓ 约束满足')

print('\n 选中单品清单（前 10 个）：')

print(f'    {"单品名称":<15} {"日均销量(kg)":>12} {"损耗率":>10}')

print('    ' + '-' * 45)

for _, row in selected_info.head(10).iterrows():

    name = row['单品名称']

    if len(name) > 12:

        name = name[:10] + '...'

    print(f'    {name:<15} {row["日均销量"]:>12.2f} {row["损耗率"]*100:>9.2f}%')

if len(selected_info) > 10:

    print(f'    ... 还有{len(selected_info) - 10}个单品')

# ----- 图 6：选中单品销量排名 -----

fig6, ax6 = plt.subplots(figsize=(14, 8))

top_items = selected_info.nlargest(15, '日均销量')

bars = ax6.barh(top_items['单品名称'], top_items['日均销量'],
color='#45B7D1')

ax6.set_xlabel('日均销量 (kg)')

ax6.set_ylabel('单品名称')

ax6.set_title('图 6：选中单品中销量最高的 15 个单品')

```

```

for bar, val in zip(bars, top_items['日均销量']):

    ax6.text(val + 0.3, bar.get_y() + bar.get_height()/2,

             f'{val:.1f}kg', va='center', fontsize=9)

plt.tight_layout()

plt.savefig('图 6_选中单品销量排名.png', dpi=150, bbox_inches='tight')

plt.close()

print('    ✓ 已保存：图 6_选中单品销量排名.png')

# ----- 图 7：选中单品品类分布 -----

fig7, ax7 = plt.subplots(figsize=(10, 6))

cat_counts = selected_info['分类名称'].value_counts()

bars = ax7.bar(cat_counts.index, cat_counts.values,

               color=['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A',

                    '#98D8C8', '#DDA0DD'])

ax7.set_xlabel('品类')

ax7.set_ylabel('选中单品数')

ax7.set_title('图 7：选中单品的品类分布')

for bar, val in zip(bars, cat_counts.values):

    ax7.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.3,

             f'{val}', ha='center', va='bottom', fontsize=12)

plt.tight_layout()

plt.savefig('图 7_选中单品品类分布.png', dpi=150, bbox_inches='tight')

plt.close()

```

```

print('  ✓ 已保存：图 7_选中单品品类分布.png')

print('  问题 3 完成！\n')

# ===== 6. 问题 4 =====

print('【问题 4】数据采集建议')

print('  ' + '=' * 60)

print("""
    1. 天气数据（温度、降水、日照）
        → 影响蔬菜消费需求，提升预测精度

    2. 节假日和促销活动记录
        → 区分自然销量和促销销量

    3. 缺货记录数据
        → 还原真实潜在需求

    4. 日内分时销售和折扣记录
        → 支持动态定价模型

    5. 供应商供货稳定性和产地信息
        → 预判批发价格波动

    6. 消费者画像和会员数据
        → 辅助单品筛选

""")

print('  ' + '=' * 60)

# ===== 7. 完成 =====

print('\n' + '=' * 60)

```

```
print('  程序运行完成！ ')\n\nprint(f'  使用数据： {len(sales_df)}条销售记录')\n\nprint(f'  共生成 7 张图片： ')\n\nprint('    图 1_各品类总销量.png')\n\nprint('    图 2_品类销量分布.png')\n\nprint('    图 3_品类相关性热力图.png')\n\nprint('    图 4_品类补货量与定价.png')\n\nprint('    图 5_品类每日收益.png')\n\nprint('    图 6_选中单品销量排名.png')\n\nprint('    图 7_选中单品品类分布.png')\n\nprint('=' * 60)\n\n# 打开文件夹\n\nimport os\n\nos.startfile(os.getcwd())\n\nprint('✔ 已打开文件夹！ ')
```